

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: Database Management Systems
COURSE CODE: CSA0501
NAME: M.NITHYASHREENARAYANI
REG.NO: 192521416

COURSE OUTCOME COVERED

CO2: *Apply the relational model, relational algebra, SQL query structures, and views to solve database querying and data manipulation problems.* (BL3, BL6)

Activity Tool : Scenario-Based Task (High)

Assessment Tool Weightage: 35%

Dr

QUESTIONS			Total Marks: 50
Q.No	Question	Bloom's Level	Marks
1	Given the scenario of a School Management System, identify relations and write SQL to list all students with their class teacher and grade.	BL3 – Apply	5
2	A hospital wants to track patient appointments. Design the relational schema and write SQL to find doctors with more than 10 appointments this month.	BL3 – Apply	5
3	In an e-commerce scenario, write SQL queries using sub-queries to find products that have never been ordered.	BL3 – Apply	5
4	For a university database, write relational algebra expressions to find students enrolled in both 'DBMS' and 'OS' courses.	BL3 – Apply	5
5	Given a payroll scenario, create a view showing employees whose salary is below department average and write a query to update their salary by 10%.	BL6 – Create	5
6	In a banking scenario, apply JOIN operations to retrieve customer account details, transaction history, and branch information.	BL3 – Apply	5

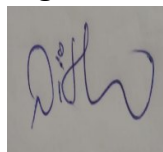
7	A retail store needs daily sales summary. Write SQL with GROUP BY and HAVING to report total sales per category exceeding Rs. 10,000.	BL3 – Apply	5
8	For a library system scenario, construct tuple relational calculus expressions to find members who borrowed books from all genres.	BL3 – Apply	5
9	Given an inventory management scenario, define CHECK and FOREIGN KEY constraints and demonstrate their enforcement through SQL.	BL3 – Apply	5
10	Design a relational schema for an HR system and create materialized views for monthly attendance and performance summaries.	BL6 – Create	5

Code of Conduct:

I M.NITHYASHREENARAYANI / 192521416 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



RUBRICS FOR CALCULATION:

Scenario based assessment					
Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Scenario	20%	Demonstrates clear and complete understanding of the scenario and context	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Misinterprets or fails to understand the scenario
Identification of Key Issues	20%	Accurately identifies all relevant issues and factors involved	Identifies most key issues with minor omissions	Identifies limited issues; some irrelevant points	Unable to identify key issues

Application of Concepts	25%	Applies appropriate concepts effectively to address the scenario	Applies concepts with minor errors	Limited or partially correct application	Unable to apply relevant concepts
Decision Making	20%	Makes well-reasoned, logical decisions with strong rationale	Makes reasonable decisions with some justification	Makes weak decisions with limited justification	No clear decisions made or rationale provided
Clarity of Response	15%	Response is clear, structured, and logically presented	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand

DBMS Assessment

Q1. School Management System — Students with Class Teacher & Grade (BL3 – Apply)

Relations (Schema)

Student (student_id PK, name, dob, class_id FK -> Class.class_id)

Class (class_id PK, grade, section, teacher_id FK -> Teacher.teacher_id)

Teacher (teacher_id PK, name, subject)

Relationships:

- Teacher (1) — (M) Class → one teacher can be class-teacher of one or more classes.
- Class (1) — (M) Student → one class has many students.

DDL

```
CREATE TABLE Teacher (  
    teacher_id INT PRIMARY KEY,  
    name      VARCHAR(50) NOT NULL,  
    subject   VARCHAR(30)  
);
```

```
CREATE TABLE Class (  
    class_id INT PRIMARY KEY,  
    grade    VARCHAR(10) NOT NULL,  
    section  VARCHAR(5),  
    teacher_id INT,  
    FOREIGN KEY (teacher_id) REFERENCES Teacher(teacher_id)  
);
```

```
CREATE TABLE Student (  
    student_id INT PRIMARY KEY,  
    name       VARCHAR(50) NOT NULL,
```

```

dob      DATE,
class_id INT,
FOREIGN KEY (class_id) REFERENCES Class(class_id)
);

```

Query — List all students with class teacher and grade

```

SELECT s.student_id,
       s.name      AS student_name,
       c.grade,
       c.section,
       t.name      AS class_teacher
FROM   Student s
JOIN   Class c ON s.class_id = c.class_id
JOIN   Teacher t ON c.teacher_id = t.teacher_id
ORDER BY c.grade, s.name;

```

Q2. Hospital — Patient Appointments (BL3 – Apply)

Relations (Schema)

Patient (patient_id PK, name, dob, phone)

Doctor (doctor_id PK, name, specialization)

Appointment (appointment_id PK, patient_id FK, doctor_id FK,
appt_date DATE, appt_time TIME, status)

Relationships:

- Doctor (1) — (M) Appointment
- Patient (1) — (M) Appointment
- (Doctor M — M Patient, realized through Appointment as the associative/bridge entity)

DDL

```

CREATE TABLE Patient (
    patient_id INT PRIMARY KEY,

```

```
name    VARCHAR(50) NOT NULL,  
dob     DATE,  
phone   VARCHAR(15)  
);
```

```
CREATE TABLE Doctor (  
    doctor_id    INT PRIMARY KEY,  
    name         VARCHAR(50) NOT NULL,  
    specialization VARCHAR(40)  
);
```

```
CREATE TABLE Appointment (  
    appointment_id INT PRIMARY KEY,  
    patient_id INT,  
    doctor_id INT,  
    appt_date DATE NOT NULL,  
    appt_time TIME,  
    status   VARCHAR(15) DEFAULT 'Scheduled',  
    FOREIGN KEY (patient_id) REFERENCES Patient(patient_id),  
    FOREIGN KEY (doctor_id) REFERENCES Doctor(doctor_id)  
);
```

Query — Doctors with more than 10 appointments this month

```
SELECT d.doctor_id,  
       d.name,  
       COUNT(a.appointment_id) AS appointment_count  
FROM   Doctor d  
JOIN   Appointment a ON d.doctor_id = a.doctor_id  
WHERE  MONTH(a.appt_date) = MONTH(CURRENT_DATE)  
       AND YEAR(a.appt_date) = YEAR(CURRENT_DATE)
```

```
GROUP BY d.doctor_id, d.name
HAVING COUNT(a.appointment_id) > 10
ORDER BY appointment_count DESC;
```

Q3. E-Commerce — Products Never Ordered (Sub-queries) (BL3 – Apply)

Relevant Relations

Product (product_id PK, name, price, category)

OrderItem (order_item_id PK, order_id FK, product_id FK, qty)

Query 1 — Using NOT IN

```
SELECT product_id, name, price
FROM Product
WHERE product_id NOT IN (
    SELECT product_id
    FROM OrderItem
    WHERE product_id IS NOT NULL
);
```

Query 2 — Using NOT EXISTS (safer, NULL-proof)

```
SELECT p.product_id, p.name, p.price
FROM Product p
WHERE NOT EXISTS (
    SELECT 1
    FROM OrderItem oi
    WHERE oi.product_id = p.product_id
);
```

Query 3 — Using LEFT JOIN / IS NULL (alternative, not a sub-query but often asked as a variant)

```
SELECT p.product_id, p.name
FROM Product p
LEFT JOIN OrderItem oi ON p.product_id = oi.product_id
```

WHERE oi.product_id IS NULL;

NOT EXISTS is generally preferred over NOT IN when the sub-query column may contain NULLs.

Q4. University Database — Relational Algebra (Students in Both DBMS & OS) (BL3 – Apply)

Relations

Student (sid, sname, ...)

Course (cid, cname, ...)

Enrolled (sid, cid)

Step 1 — Students enrolled in DBMS

$R1 = \pi_{sid} (\sigma_{cname='DBMS'} (Enrolled \bowtie Course))$

Step 2 — Students enrolled in OS

$R2 = \pi_{sid} (\sigma_{cname='OS'} (Enrolled \bowtie Course))$

Step 3 — Intersection (enrolled in BOTH)

Result = $R1 \cap R2$

Full expression in one line

$\pi_{sid} (\sigma_{cname='DBMS'} (Enrolled \bowtie Course)) \cap \pi_{sid} (\sigma_{cname='OS'} (Enrolled \bowtie Course))$

Equivalent SQL (for cross-verification)

SELECT e1.sid

FROM Enrolled e1

JOIN Course c1 ON e1.cid = c1.cid AND c1.cname = 'DBMS'

WHERE e1.sid IN (

 SELECT e2.sid

 FROM Enrolled e2

 JOIN Course c2 ON e2.cid = c2.cid AND c2.cname = 'OS'

);

Q5. Payroll — View for Below-Average Salary + Bulk Update (BL6 – Create)

Relations

Employee (emp_id PK, name, dept_id FK, salary)

Department (dept_id PK, dept_name)

Step 1 — Create the view

```
CREATE VIEW BelowAvgSalaryEmp AS
```

```
SELECT e.emp_id, e.name, e.dept_id, e.salary
```

```
FROM Employee e
```

```
WHERE e.salary < (
```

```
    SELECT AVG(e2.salary)
```

```
    FROM Employee e2
```

```
    WHERE e2.dept_id = e.dept_id
```

```
);
```

Step 2 — Update salary of those employees by 10%

```
UPDATE Employee
```

```
SET salary = salary * 1.10
```

```
WHERE emp_id IN (SELECT emp_id FROM BelowAvgSalaryEmp);
```

Note: Recompute/refresh the view logically before re-checking, since department averages shift once salaries change — run the UPDATE only once per cycle, not repeatedly against the same view instance.

Q6. Banking — JOINS Across Customer, Account, Transaction, Branch (BL3 – Apply)

Relations

Customer (cust_id PK, name, address)

Branch (branch_id PK, branch_name, city)

Account (acc_no PK, cust_id FK, branch_id FK, acc_type, balance)

Transaction (txn_id PK, acc_no FK, txn_date, txn_type, amount)

Query — Customer + Account + Transaction + Branch details

```
SELECT c.cust_id,
```

```
       c.name      AS customer_name,
```

```
       a.acc_no,
```

```

a.acc_type,
a.balance,
b.branch_name,
b.city,
t.txn_id,
t.txn_date,
t.txn_type,
t.amount
FROM Customer c
JOIN Account a ON c.cust_id = a.cust_id
JOIN Branch b ON a.branch_id = b.branch_id
JOIN Transaction t ON a.acc_no = t.acc_no
ORDER BY c.cust_id, t.txn_date DESC;

```

Use LEFT JOIN on Transaction instead of JOIN if accounts with **no** transactions should still appear.

Q7. Retail — Daily Sales Summary (GROUP BY + HAVING) (BL3 – Apply)

Relations

Sales (sale_id PK, product_id FK, category, sale_date, amount)

Query — Total sales per category exceeding Rs. 10,000 (today)

```

SELECT category,
       SUM(amount) AS total_sales
FROM Sales
WHERE sale_date = CURRENT_DATE
GROUP BY category
HAVING SUM(amount) > 10000
ORDER BY total_sales DESC;

```

Q8. Library System — Tuple Relational Calculus (Members Who Borrowed from All Genres) (BL3 – Apply)

Relations

Member (mid, mname, ...)

Book (bid, title, genre)

Borrow (mid, bid, borrow_date)

TRC Expression (division-style / universal quantification)

$$\{ t \mid t \in \text{Member} \wedge \\ \forall g (g \in (\pi_{\text{genre}}(\text{Book})) \rightarrow \\ \exists b (b \in \text{Book} \wedge b.\text{genre} = g.\text{genre} \wedge \\ \exists r (r \in \text{Borrow} \wedge r.\text{mid} = t.\text{mid} \wedge r.\text{bid} = b.\text{bid}))) \}$$

In words: select member tuples t such that for every genre g that exists in Book, there exists a book b of that genre and a Borrow record r linking t (the member) to b — i.e., the member has borrowed at least one book from every genre.

Equivalent SQL (relational division), for verification

```
SELECT DISTINCT m.mid, m.mname
FROM   Member m
WHERE  NOT EXISTS (
    SELECT DISTINCT genre FROM Book
    EXCEPT
    SELECT DISTINCT bk.genre
    FROM   Borrow br
    JOIN   Book bk ON br.bid = bk.bid
    WHERE  br.mid = m.mid
);
```

Q9. Inventory Management — CHECK & FOREIGN KEY Constraints (BL3 – Apply)

Relations

Supplier (supplier_id PK, name)

Product (product_id PK, product_name, quantity, reorder_level,
unit_price, supplier_id FK)

DDL with constraints

```
CREATE TABLE Supplier (  
    supplier_id INT PRIMARY KEY,  
    name      VARCHAR(50) NOT NULL  
);
```

```
CREATE TABLE Product (  
    product_id  INT PRIMARY KEY,  
    product_name VARCHAR(50) NOT NULL,  
    quantity    INT NOT NULL CHECK (quantity >= 0),  
    reorder_level INT NOT NULL CHECK (reorder_level >= 0),  
    unit_price  DECIMAL(10,2) CHECK (unit_price > 0),  
    supplier_id INT,  
    CONSTRAINT fk_supplier  
        FOREIGN KEY (supplier_id) REFERENCES Supplier(supplier_id)  
        ON DELETE SET NULL  
        ON UPDATE CASCADE  
);
```

Demonstrating enforcement

a) CHECK constraint violation (negative quantity) — rejected:

```
INSERT INTO Product VALUES (101, 'Steel Rod', -5, 10, 250.00, 1);  
-- ERROR: CHECK constraint 'quantity >= 0' violated
```

b) FOREIGN KEY violation (non-existent supplier) — rejected:

```
INSERT INTO Product VALUES (102, 'Cement Bag', 100, 20, 350.00, 999);  
-- ERROR: FOREIGN KEY constraint fails, supplier_id 999 does not exist in Supplier
```

c) Valid insert — accepted:

```
INSERT INTO Supplier VALUES (1, 'ABC Traders');  
INSERT INTO Product VALUES (103, 'Cement Bag', 100, 20, 350.00, 1);  
-- SUCCESS
```

d) ON DELETE SET NULL demonstration:

DELETE FROM Supplier WHERE supplier_id = 1;

-- Product.supplier_id for product 103 is automatically set to NULL

Q10. HR System — Schema + Materialized Views for Attendance & Performance (BL6 – Create)

Relational Schema

Employee (emp_id PK, name, dept_id FK, designation, doj)

Department (dept_id PK, dept_name)

Attendance (att_id PK, emp_id FK, att_date, status) -- status: Present/Absent/Leave

Performance (perf_id PK, emp_id FK, review_month, rating, remarks)

DDL (core tables)

CREATE TABLE Department (

dept_id INT PRIMARY KEY,

dept_name VARCHAR(40)

);

CREATE TABLE Employee (

emp_id INT PRIMARY KEY,

name VARCHAR(50) NOT NULL,

dept_id INT,

designation VARCHAR(40),

doj DATE,

FOREIGN KEY (dept_id) REFERENCES Department(dept_id)

);

CREATE TABLE Attendance (

att_id INT PRIMARY KEY,

emp_id INT,

att_date DATE,

```
status VARCHAR(10) CHECK (status IN ('Present','Absent','Leave')),  
FOREIGN KEY (emp_id) REFERENCES Employee(emp_id)  
);
```

```
CREATE TABLE Performance (  
    perf_id INT PRIMARY KEY,  
    emp_id INT,  
    review_month DATE,  
    rating DECIMAL(3,1) CHECK (rating BETWEEN 0 AND 10),  
    remarks VARCHAR(200),  
    FOREIGN KEY (emp_id) REFERENCES Employee(emp_id)  
);
```

Materialized View — Monthly Attendance Summary

```
CREATE MATERIALIZED VIEW MonthlyAttendanceSummary AS  
SELECT e.emp_id,  
       e.name,  
       DATE_TRUNC('month', a.att_date) AS month,  
       COUNT(*) FILTER (WHERE a.status = 'Present') AS days_present,  
       COUNT(*) FILTER (WHERE a.status = 'Absent') AS days_absent,  
       COUNT(*) FILTER (WHERE a.status = 'Leave') AS days_leave,  
       COUNT(*) AS total_marked_days  
FROM Employee e  
JOIN Attendance a ON e.emp_id = a.emp_id  
GROUP BY e.emp_id, e.name, DATE_TRUNC('month', a.att_date);
```

Materialized View — Monthly Performance Summary

```
CREATE MATERIALIZED VIEW MonthlyPerformanceSummary AS  
SELECT e.emp_id,  
       e.name,
```

```

    d.dept_name,
    p.review_month,
    AVG(p.rating) AS avg_rating,
    MAX(p.rating) AS best_rating,
    MIN(p.rating) AS worst_rating
FROM   Employee e
JOIN   Department d ON e.dept_id = d.dept_id
JOIN   Performance p ON e.emp_id = p.emp_id
GROUP BY e.emp_id, e.name, d.dept_name, p.review_month;

```

Refreshing the materialized views (PostgreSQL/Oracle)

```

REFRESH MATERIALIZED VIEW MonthlyAttendanceSummary;
REFRESH MATERIALIZED VIEW MonthlyPerformanceSummary;

```

MySQL emulation (no native MATERIALIZED VIEW support)

```

CREATE TABLE MonthlyAttendanceSummary AS
SELECT  e.emp_id, e.name,
        DATE_FORMAT(a.att_date, '%Y-%m-01') AS month,
        SUM(a.status = 'Present') AS days_present,
        SUM(a.status = 'Absent') AS days_absent,
        SUM(a.status = 'Leave') AS days_leave
FROM    Employee e
JOIN    Attendance a ON e.emp_id = a.emp_id
GROUP BY e.emp_id, e.name, month;

```

-- Refresh periodically via a scheduled EVENT:

```

CREATE EVENT refresh_attendance_summary
ON SCHEDULE EVERY 1 DAY
DO
BEGIN
    TRUNCATE TABLE MonthlyAttendanceSummary;

```

```
INSERT INTO MonthlyAttendanceSummary
SELECT e.emp_id, e.name, DATE_FORMAT(a.att_date, '%Y-%m-01'),
       SUM(a.status = 'Present'), SUM(a.status = 'Absent'), SUM(a.status = 'Leave')
FROM Employee e JOIN Attendance a ON e.emp_id = a.emp_id
GROUP BY e.emp_id, e.name, DATE_FORMAT(a.att_date, '%Y-%m-01');
END;
```